关于CFG的研究

原创 于 2021-04-03 11:39:09 发布 · 2k 阅读 · 1 · 8 · CC 4.0 BY-SA版权

文章标签： #Windows #CFG保护机制 #逆向

关于CFG的研究

CFG（Control Flow Guard）控制流防护，是微软在Windows8.0以及Windows10上推出的新的防护机制

防护点在于，防护间接调用，防止在程序间接调用函数的时候，使用恶意代码进行替换，导致执行恶意程序。CFG的实现机制在于每当存在间接调用的函数的时候，就会先去判断一下间接调用的地址处是不是一个有效的函数的起始地址。主要关注缓解间接调用和调用不可靠目

标的问题(在没有CFG支持的Windows 中，这个函数不做任何事。在Windows 10中，有了CFG的支持，它指向ntdll!LdrpValidateUserCallTarget函数)

> 加了CFG和不加CFG的程序，反汇编的对比

```
call    operator_new__
add     esp, 4
mov     [ebp+var_C], eax
mov     eax, [ebp+var_C]
mov     [ebp+var_8], eax
mov     [ebp+var_4], 0
jmp     short loc_4010D2
```

```
loc_4010D2:
cmp     [ebp+var_4], 5
jge     short loc_4010E7
```

```
+var_4]
+var_8]
[eax+edx*4], offset ?foo@@YAHH@Z ; foo(int)
_4010C9
```

```
loc_4010E7:
push    1
mov     ecx, 4
imul    edx, ecx, 0
mov     eax, [ebp+var_8]
mov     ecx, [eax+edx]
call    ecx
add     esp, 4
xor     eax, eax
mov     esp, ebp
pop     ebp
retn
_main endp
```

通过偏移间接调用

https://blog.csdn.net/Xr__nc

```
call    operator_new__
add     esp, 4
mov     [ebp+var_10], eax
mov     eax, [ebp+var_10]
mov     [ebp+var_8], eax
mov     [ebp+var_4], 0
jmp     short loc_4010D2
```

```
loc_4010D2:
cmp     [ebp+var_4], 5
jge     short loc_4010E7
```

```
_4]
_8]
x+edx*4], offset ?foo@@YAHH@Z ; foo(int)
0C9
```

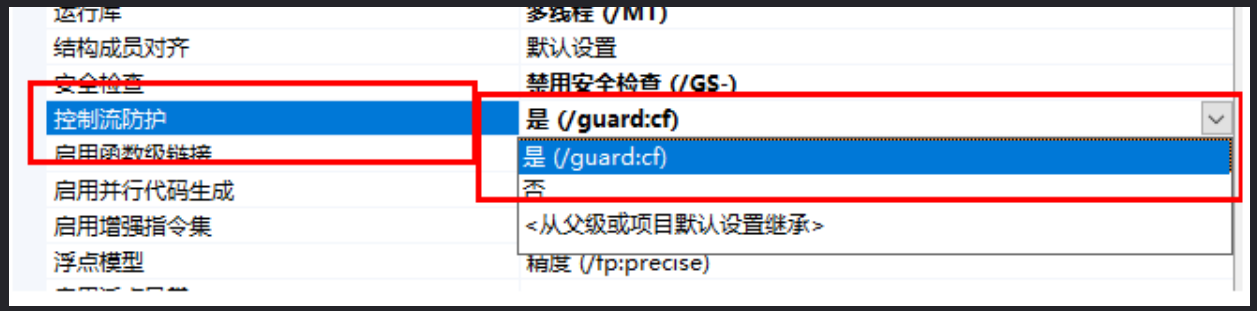
```
loc_4010E7:
push    1
mov     ecx, 4
imul    edx, ecx, 0
mov     eax, [ebp+var_8]
mov     ecx, [eax+edx]
mov     [ebp+var_C], ecx
mov     ecx, [ebp+var_C]
call    ds:__guard_check_icall_fptr
call    [ebp+var_C]
add     esp, 4
xor     eax, eax
mov     esp, ebp
pop     ebp
retn
_main endp
```

添加CFG防护机制

https://blog.csdn.net/sxr__nc

通过对比二者反汇编结果可以明显的看出加了CFG防护机制的程序，会在调用之前，调用_guard_check_icall_fptr函数，该函数的功能其实就是对目标偏移进行校验，检查其是否是一个有效的函数起始地址

注意：给程序添加CFG防护只需要在编译生成的时候，添加选项就好，如下图所示



另外，还有一点需要注意，就是在生成的时候，要把编译器的优化选项关闭掉，否则生成的代码通过IDA反编译的结构也是看不到CFG守护函数的

添加CFG防护之后，检测具体有没有添加，可以使用VS自带的工具dumpbin.exe 执行命令

```
dumpbin.exe /headers /loadconfig C:\Users\Administrator\source\repos\LoadDestDLL\Release\LoadDestDLL.exe
```

```
OPTIONAL HEADER VALUES
             10B magic # (PE32)
           14.16 linker version
           10C00 size of code
            8E00 size of initialized data
               0 size of uninitialized data
            1380 entry point (00401380) _mainCRTStartup
            1000 base of code
           12000 base of data
          400000 image base (00400000 to 0041CFFF)
            1000 section alignment
             200 file alignment
            6.00 operating system version
            0.00 image version
            6.00 subsystem version
               0 Win32 version
           1D000 size of image
             400 size of headers
               0 checksum
               3 subsystem (Windows CUI)
            C140 DLL characteristics
                   Dynamic base
                   NX compatible
                   Control Flow Guard
                   Terminal Server Aware
```

开启了CFG防护

https://blog.csdn.net/sxr__nc

```
Section contains the following load config:

          000000A0 size
                 0 time date stamp
              0.00 Version
                 0 GlobalFlags Clear
                 0 GlobalFlags Set
                 0 Critical Section Default Timeout
                 0 Decommit Free Block Threshold
                 0 Decommit Total Free Threshold
          00000000 Lock Prefix Table
                 0 Maximum Allocation Size
                 0 Virtual Memory Threshold
                 0 Process Heap Flags
                 0 Process Affinity Mask
                 0 CSD Version
              0000 Dependent Load Flag
          00000000 Edit List
          00419018 Security Cookie
          00417870 Safe Exception Handler Table
                 3 Safe Exception Handler Count
          0041211C Guard CF address of check-function pointer
          00000000 Guard CF address of dispatch-function pointer
          00412164 Guard CF function table
                38 Guard CF function count
          10017500 Guard Flags
```

CFG函数表地址

CFG函数个数

https://blog.csdn.net/sxr__nc

```
0041211C Guard CF address of check-function pointer
```

Guard CF address of check-function pointer：该函数地址指向函数_guard_check_icall_fptr

动态调试，Windbg动态调试EXE文件方法（断到入口点的方法）

1：加载符号，设置符号路径

2：File->OpenProcess

3：打开文件之后，停在Ntdll.dll模块里边，这个时候，可以使用lm命令来查看，当前进程的一些起始信息：

```
0:000:x86> lm
start             end                 module name
012a0000 012bd000   LoadDestDLL C (private pdb symbols)  C:\Users\sam\Desktop\LoadDestDLL.pdb
735d0000 735d8000   wow64cpu   (deferred)
735e0000 7363c000   wow64win   (deferred)
73640000 7367f000   wow64      (export symbols)       C:\Windows\SYSTEM32\wow64.dll
758d0000 759e0000   kernel32   (export symbols)       C:\Windows\syswow64\kernel32.dll
771d0000 77216000   KERNELBASE   (deferred)
77530000 776d9000   ntdll      (export symbols)       C:\Windows\SYSTEM32\ntdll.dll
```

之后可以使用 以下命令来解析PE文件

> !dh -a(-f) 目标模块的文件名

获取到入口点地址之后，加基址下断点：

```
0:000:x86> !dh -f LoadDestDLL

File Type: EXECUTABLE IMAGE
FILE HEADER VALUES
     14C machine (i386)
       5 number of sections
5F180C68 time date stamp Wed Jul 22 17:52:40 2020

       0 file pointer to symbol table
       0 number of symbols
      E0 size of optional header
     102 characteristics
            Executable
            32 bit word machine

OPTIONAL HEADER VALUES
     10B magic #
   14.16 linker version
   10C00 size of code
    8E00 size of initialized data
       0 size of uninitialized data
    1380 address of entry point
    1000 base of code
```

https://blog.csdn.net/sxr__nc

下bp断点，之后执行就会断到入口点，但是需要注意入口点 不是main函数的入口点，这个直接在IDA里边看到之后，修改基址跳过去下断点就好了

到达目标函数之后,单步步进:

```
10e4 00eb            add     bl,ch
10e6 e26a            loop    LoadDestDLL!pre_c_initialization+0x32 (00f11152)
10e8 01b904000000    add     dword ptr [ecx+4],edi
10ee 6bd100          imul    edx,ecx,0
10f1 8b45f8          mov     eax,dword ptr [ebp-8]
10f4 8b0c10          mov     ecx,dword ptr [eax+edx]
10f7 894df4          mov     dword ptr [ebp-0Ch],ecx
10fa 8b4df4          mov     ecx,dword ptr [ebp-0Ch]
10fd ff151c21f200    call    dword ptr [LoadDestDLL!__guard_check_icall_fptr (00f2211c)]
1103 ff55f4          call    dword ptr [ebp-0Ch]
1106 83c404          add     esp,4
1109 33c0            xor     eax,eax
110b 8be5            mov     esp,ebp
110d 5d              pop     ebp
110e c3              ret
DestDLL!operator new[]:
110f 55              push    ebp
```

单步步进

https://blog.csdn.net/sxr__nc

步进之后:整个CFG的处理过程(这里需要注意,在调试的时候选取支持CFG机制的Windows版本才会存在相应的代码块,否则什么也不做直接返回)

```
77838beb 5d              pop     ebp
77838bec 5b              pop     ebx
77838bed c21400          ret     14h
77838bf0 8b15f8128d77    mov     edx,dword ptr [ntdll_777b0000!LdrSystemDllInitBlock+0xb0 (778d12f8)] ds:002b:778d12f8=00f30000
77838bf6 8bc1            mov     eax,ecx
77838bf8 c1e808          shr     eax,8
77838bfb 8b1482          mov     edx,dword ptr [edx+eax*4]
77838bfe 8bc1            mov     eax,ecx
77838c00 c1e803          shr     eax,3
77838c03 f6c10f          test    cl,0Fh
77838c06 7506            jne     ntdll_777b0000!RtlInterlockedCompareExchange64+0x3e (77838c0e)
77838c08 0fa3c2          bt      edx,eax
```

https://blog.csdn.net/sxr__nc

ds	2b
edi	3641a48
esi	f29e50
ebx	3043000
edx	f30000
ecx	f11080
eax	f110
ebp	32ffa0c
eip	77838bfb
cs	23

存储目标偏移地址，也就是间接调用的函数地址

```
77838bf0 8b15f8128d77    mov     edx,dword ptr [ntdll_777b0000!LdrSystemDllInitBlock+0xb0 (778d12f8)]
77838bf6 8bc1            mov     eax,ecx
77838bf8 c1e808          shr     eax,8
77838bfb 8b1482          mov     edx,dword ptr [edx+eax*4]
77838bfe 8bc1            mov     eax,ecx
77838c00 c1e803          shr     eax,3
77838c03 f6c10f          test    cl,0Fh
77838c06 7506            jne     ntdll_777b0000!RtlInterlockedCompareExchange64+0x3e (77838c0e)
77838c08 0fa3c2          bt      edx,eax
77838c0b 730a            jae     ntdll_777b0000!RtlInterlockedCompareExchange64+0x47 (77838c17)
77838c0d c3              ret
77838c0e 0fbaf000        btr     eax,0
77838c12 0fa3c2          bt      edx,eax
77838c15 7309            jae     ntdll_777b0000!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c17 83c801          or      eax,1
77838c1a 0fa3c2          bt      edx,eax
77838c1d 7301            jae     ntdll_777b0000!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c1f c3              ret
```

https://blog.csdn.net/sxr__nc

操作系统在创建支持CFG机制的进程的时候,将CFGbitmap映射到该进程的地址空间中,同时将基址保存在ntdll!LdrSystemDllInitBlock+0xb0里边

(绿盟发表的文章里边,这个位置的基址存储位置是ntdll!LdrSystemDllInitBlock+0x60,但是我在实际调试的过程中,看到的是0xb0),**但是在Windows Internal的文章里边标明确实是0x60**

```
77838beb 5d          pop     ebp
77838bec 5b          pop     ebx
77838bed c21400      ret     14h
77838bf0 8b15f8128d77  mov   edx,dword ptr [ntdll_777b0000!LdrSystemDllInitBlock+0xb0 (778d12f8)] ds
77838bf6 8bc1        mov     eax,ecx
77838bf8 c1e808      shr     eax,8
77838bfb 8b1482      mov     edx,dword ptr [edx+eax*4]
77838bfe 8bc1        mov     eax,ecx
77838c00 c1e803      shr     eax,3
77838c03 f6c10f      test    cl,0Fh
77838c06 7506        jne     ntdll_777b0000!RtlInterlockedCompareExchange64+0x3e (77838c0e)
77838c08 0fa3c2      bt      edx,eax
```

存储着CFG的BitMap数据

Windows Internal里关于该结构的描述:

Initialize LdrSystemDllInitBlock

- +0x60 : Bitmap Address
- +0x68 : Bitmap Size
- Initialized by PspPrepareSystemDllInitBlock
  - NtCreateUserProcess->PspAllocateProcess->PspSetupUserProcessAddressSpace

- Initialize LdrSystemDllInitBlock
  - +0x60 : Bitmap Address
  - +0x68 : Bitmap Size
  - Initialized by PspPrepareSystemDllInitBlock
    - NtCreateUserProcess->PspAllocateProcess->PspSetupUserProcessAddressSpace

偏移地址

CFG进行校验的方式:

CFG进行校验主要依靠一个CFGBitmap的数据,而这个数据的来源就是我们在开启了CFG防护机制的时候,会将所有间接调用的函数形成一个CFG Function Table的结构,而这个函数数据表以BitMap的方式存储在CFGBitmap里边,以供接下来的函数地址验证.操作喜用在加载支持CFG的进程的时候，根据其Guard CF Function Table来更新CFG Bitmap中该模块所对应的位。同时，将函数指针_guard_check_icall_fptr初始化为指向ntdll!LdrpValidateUserCallTarget。

BitMap的原理:

例如：定义一个int类型的数据的时候，int数据类型在内存中占4个字节的空间也就是32bit，当定义一个拥有32个int元素的数组的时候，实际占用内存空间：32*32bit，但是其实，当int类型数据的字节码的每一位都代表一个数字的话，32个数字只需要32位的空间就可以存储，如下图所示：

具体思路：

1个int占4字节即4*8=32位，那么我们只需要申请一个int数组长度为 int tmp[1+N/32]即可存储完这些数据，其中N代表要进行查找的总数，tmp中的每个元素在内存在占32位可以对应表示十进制数0~31,所以可得到BitMap表：

tmp[0]:可表示0~31

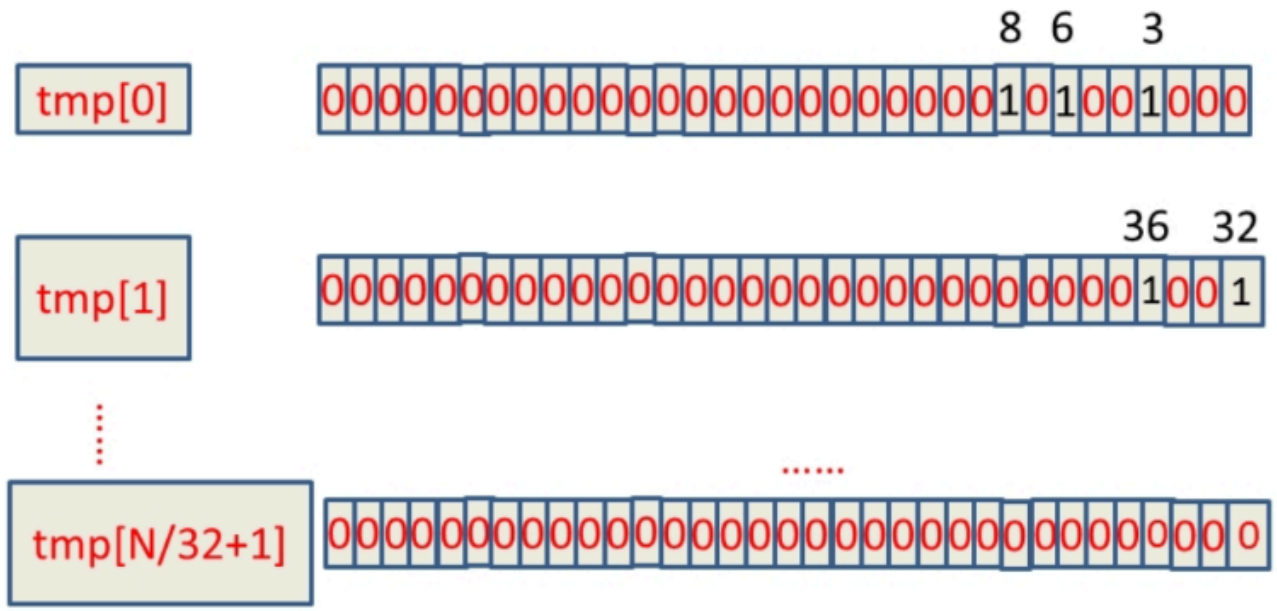
tmp[1]:可表示32~63

tmp[2]可表示64~95

.....

那么接下来就看看十进制数如何转换为对应的bit位：

假设这40亿int数据为：6,3,8,32,36,.....，那么具体的BitMap表示为：



https://blog.csdn.net/sxr__nc

CFGBitmap的逻辑也是类似于这样的。

整个CFG的校验过程:

> 1 、取目标地址的高24位作为索引
> 2 、将CFGBitmap当作一个32bit的数组，用刚才得到的索引值，在这个数组里取出一个32位的整数bits（其实就是去索引的过程）
> 3 、取刚才获得到的32位整数bits的第4-8位（3-7）作为偏移量n（定位到具体的目标地址）

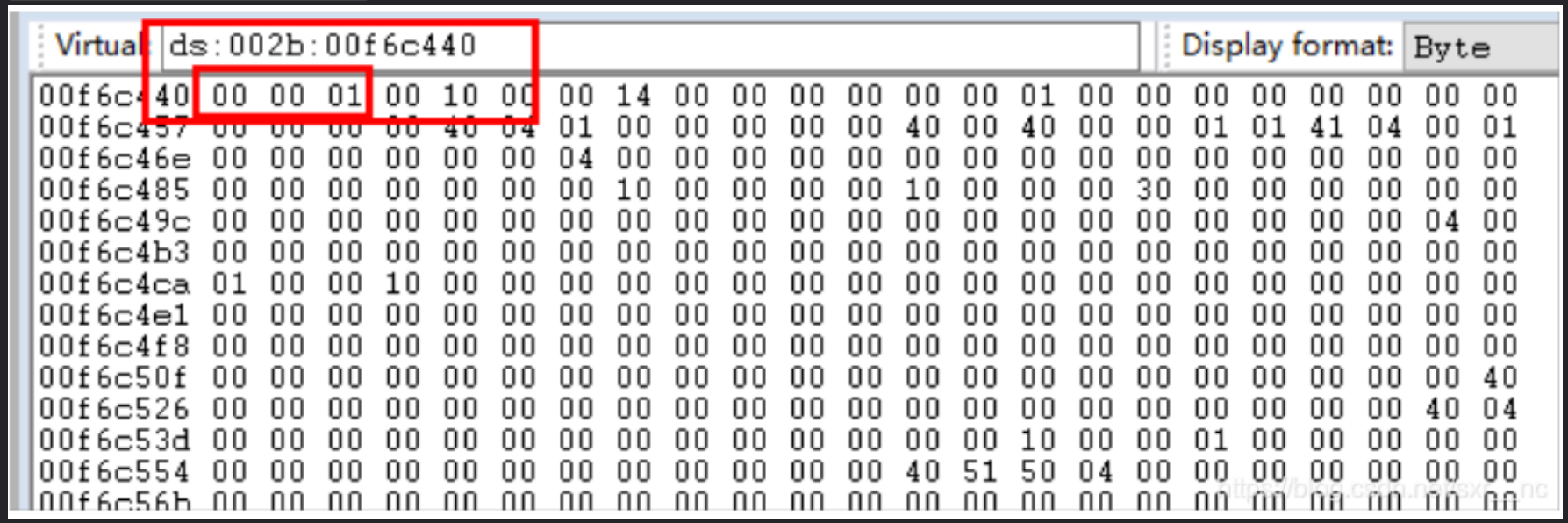
> 4 、判断目标地址是否以0x10对齐，如果不是则设置n的最低位（判断偏移量）
> 5、 取32位整数bits的第n位即为目标地址的对应位（确定目标地址）

具体验证过程：

第一步：取目标地址的高24位

`shr eax，8` 右移8位，相当于取高4个字节

`edx,dword ptr [edx+eax*4]` 取目标地址高24位作为32位数组的索引，取出32位的整数bits,获取到的索引如下图所示



第二步：

`shr eax,3` 目标地址右移3位

AI写代码 | 复制

```
eax = ecx = 0xf11080   ->   eax  >> 3 = 0x1e2210
test cl ,0Fh    //   0xa0  &  0x0F  = 0
ZF = 0           // ZF 为0 ,jne不跳转，说明以0x0f对齐
bt edx, eax     //这个过程的主要目的就是取出低位的第3-7位数据作为偏移量n
                //bt 指令把指定的位传送给CF标志寄存器，这里的意思就是把 edx 的某一位放到CF里，这个某一位由eax的值确定，那e
```

整个验证过程如上，执行bt指令之后，如果目标间接调用地址是合法的函数地址（也就是说在CFGBitmap里边存在这个函数），那么就会将cf位 置1，否则就不会将CF置位（但是其实在第一次检测没有通过的时候，系统会将eax最后一位置位，再重新检测一次）

```
77838bf8 c1e808          shr     eax,8
77838bfb 8b1482          mov     edx,dword ptr [edx+eax*4]
77838bfe 8bc1            mov     eax,ecx
77838c00 c1e803          shr     eax,3
77838c03 f6c10f          test    cl,0Fh
77838c06 7506            jne     ntdll!RtlInterlockedCompareExchange64+0x3e (77838c0e)
77838c08 0fa3c2          bt      edx,eax
77838c0b 730a            jae     ntdll!RtlInterlockedCompareExchange64+0x47 (77838c17)
77838c0d c3              ret
77838c0e 0fbaf000        btr     eax,0
77838c12 0fa3c2          bt      edx,eax
77838c15 7309            jae     ntdll!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c17 83c801          or      eax,1
77838c1a 0fa3c2          bt      edx,eax
77838c1d 7301            jae     ntdll!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c1f c3              ret
77838c20 51              push    ecx
77838c21 8d642480        lea     esp,[esp-80h]
77838c25 0f110424        movups  xmmword ptr [esp],xmm0
77838c29 0f114c2410      movups  xmmword ptr [esp+10h],xmm1
77838c2e 0f11542420      movups  xmmword ptr [esp+20h],xmm2
```

https://blog.csdn.net/sxr__nc

当不以0x10对齐的时候，执行以下分支：

```
77838bfb 8b1482          mov     edx,dword ptr [edx+eax*4]
77838bfe 8bc1            mov     eax,ecx
77838c00 c1e803          shr     eax,3
77838c03 f6c10f          test    cl,0Fh
77838c06 7506            jne     ntdll!RtlInterlockedCompareExchange64+0x3e (77838c0e)
77838c08 0fa3c2          bt      edx,eax
77838c0b 730a            jae     ntdll!RtlInterlockedCompareExchange64+0x47 (77838c17) [br=0]
77838c0d c3              ret
77838c0e 0fbaf000        btr     eax,0
77838c12 0fa3c2          bt      edx,eax
77838c15 7309            jae     ntdll!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c17 83c801          or      eax,1
77838c1a 0fa3c2          bt      edx,eax
77838c1d 7301            jae     ntdll!RtlInterlockedCompareExchange64+0x50 (77838c20)
77838c1f c3              ret
77838c20 51              push    ecx
```

https://blog.csdn.net/sxr__nc